Progress Report II

Design and Implementation of a Software Interface for Multiplexed Biosensor Control and Analysis in Clinical Diagnostics

Jan García i Girona
Bachelor's Degree in Computer Engineering
Universitat Autònoma de Barcelona

May 2026

1 Introduction

This Progress Report II presents the near-final state of the Bachelor's Thesis focused on the redesign and improvement of the software used to control, acquire, process, and visualize data from a multiplexed photonic biosensor platform at ICN2. At this stage, the project is no longer centered on initial exploration, but on presenting the implemented software changes, the current degree of functionality, the results obtained so far, and the provisional conclusions that can already be defended.

Compared with the previous delivery, this report is more direct and more technical. Greater importance is given to the real structure of the software, the actual data path from hardware to interface, the main measurable improvements already achieved, and the limitations that still remain before the final delivery.

2 Project Overview at a Glance

Table 1: Project summary

Aspect	**Summary**
Project context	Refactoring and improvement of a Python application used to operate a multiplexed photonic biosensor platform in a real laboratory environment.
Legacy software problem	Most of the logic was concentrated in a single main file, mixing GUI behavior, acquisition control, signal processing, calibration-related behavior, and session state management.
Main engineering objective	Transform the current software into a more modular, maintainable, understandable, and reusable system without breaking compatibility with the laboratory setup.
Current status	The calibration tab is functional, the measuring tab is still not fully functional, and the project is in the final integration and debugging stage.
Main visible improvements	Much faster startup time, clearer project structure, automated executable generation, and better traceability of the data flow.

3 Hardware Context and Connection Scheme

The software developed in this TFG is part of a larger experimental chain composed of the optical setup, the biosensor device, the segmented photodiode, the National Instruments acquisition hardware, and the laboratory computer where the application runs. For this reason, the software must be explained in relation to the physical system that provides the measured signal.

At a high level, the optical system produces a signal that is detected by a segmented photodiode. The photodiode generates electrical outputs associated with the measured optical behavior, and these analog signals are read by the National Instruments acquisition hardware, digitized, and made available to the Python software through the corresponding acquisition libraries. Once the data reaches the software, it is processed, transformed, calibrated when needed, and finally displayed through the graphical interface.

This context is important because many software decisions are constrained by the hardware environment. The application must remain compatible with the NI acquisition ecosystem, the driver and Python access layer used in the laboratory, and the timing constraints of real experiments. During the preparation of the new setup, the mapping of ports and connections was provided by Andrés Alonso, a doctoral student from NanoB2A's research group working on the design of this hardware setup whose support was essential to have a functional platform ready for early software testing.

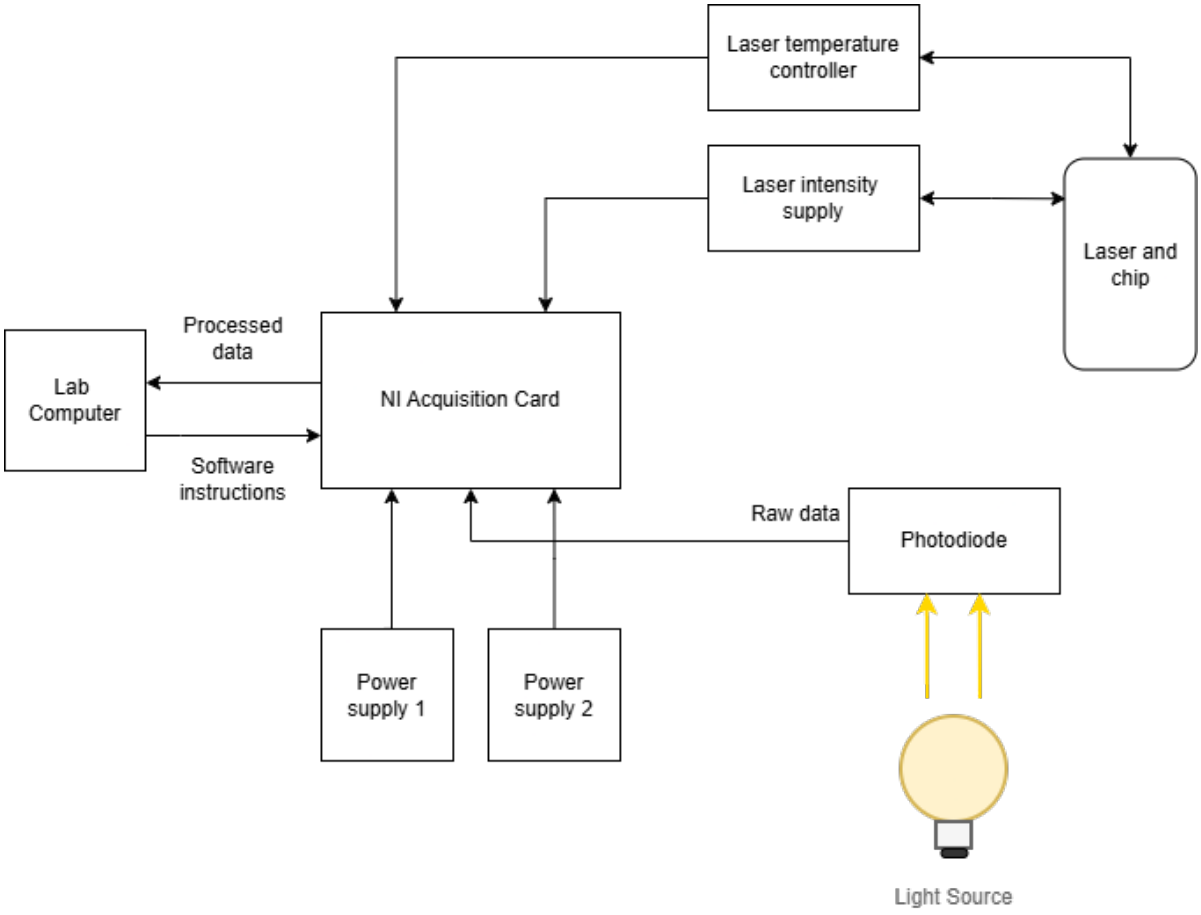

Figure 1: Hardware connection diagram of the current setup.

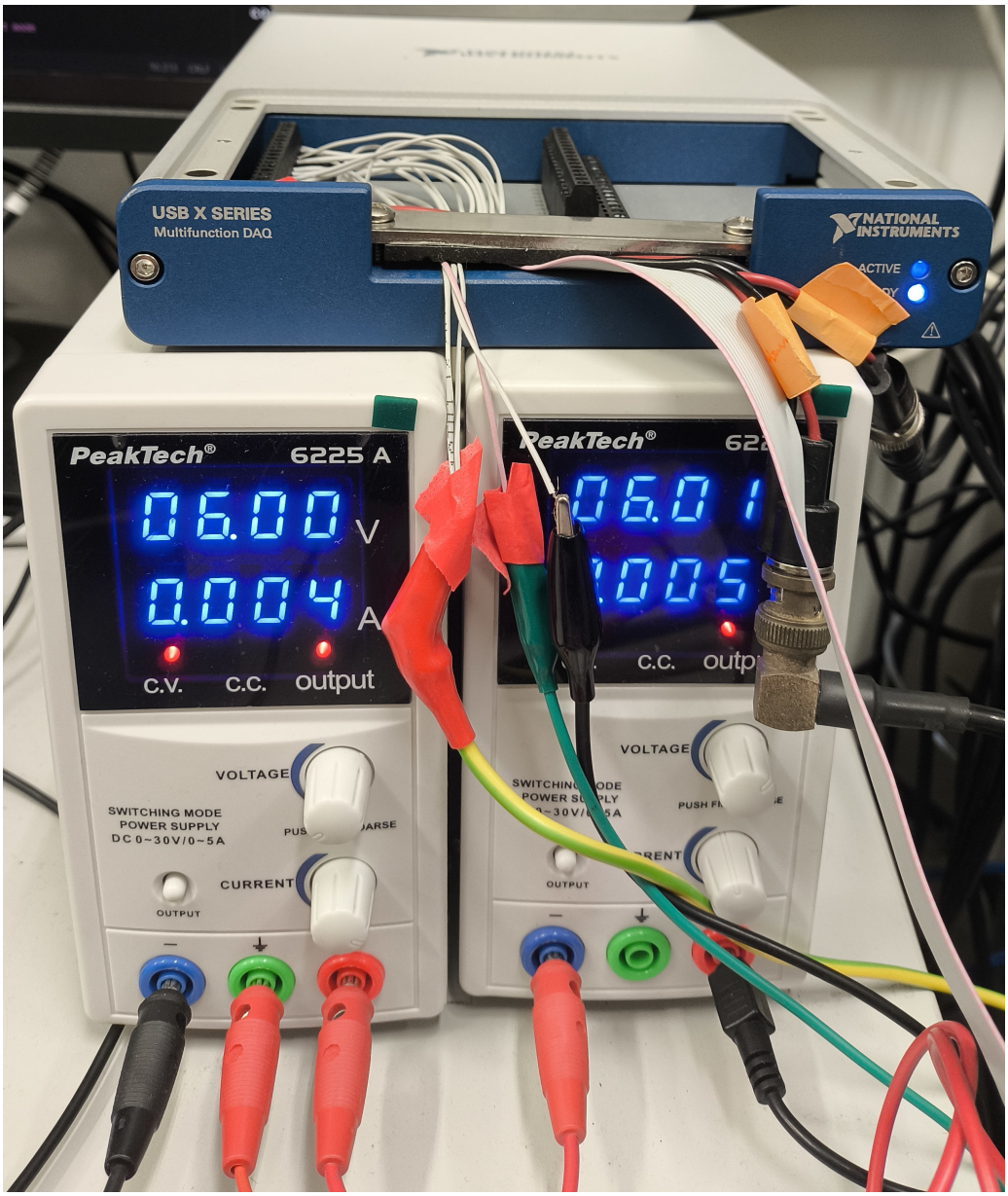

Figure 2: Photograph of the current experimental setup (NI card and power supplies) used during software development and validation.

4 Legacy Software Problem and Current Refactoring

The main structural problem of the original software is that most of the logic was concentrated in a single main file. This file mixed responsibilities that should ideally be separated, including interface behavior, acquisition control, signal-processing logic, calibration-related actions, and part of the session state management. As a result, the code was difficult to understand, difficult to modify safely, and difficult to reuse by future users.

The refactoring work performed in this TFG addresses that problem directly. Instead of continuing to build on top of the same monolithic structure, the new project reorganizes the software into clearer functional areas, separates concerns more explicitly, and makes it easier to identify where data is acquired, processed, displayed, and saved.

Table 2: Legacy software vs current refactored version

Aspect	Legacy software	Current refactored version
Core structure	Most logic concentrated in a single main file	Logic progressively separated into clearer modules and responsibilities
Startup time	Approximately 30 seconds	Approximately 2 seconds
Ease of understanding	High dependence on prior familiarity with the code	Easier to reason about due to clearer structure and workflow
Modification effort	Changes tend to affect several unrelated parts	Changes can be localized more easily
Execution method	Typically launched from the development environment	Can also be packaged as an executable for end users
Future reuse	Harder for new students or researchers to adopt	Better prepared for reuse, maintenance, and redistribution

5 Planning Follow-up and Adjustments

The initial planning proposed an incremental development process, and that general approach has been maintained. However, the actual execution required some changes once the complexity of the legacy code and the dependency on real hardware became clearer. The main adjustment was to give higher priority to architecture recovery and code restructuring before moving to full measurement functionality.

This change was justified because several parts of the original application were too tightly coupled. In practice, improving the measurement workflow, the interface logic, or the reliability of the application required first understanding and separating behavior that was previously mixed in the same execution paths. The current state of the project therefore reflects a technically coherent reordering of tasks rather than a deviation without justification.

Table 3: Follow-up of planned work

Task	Status	Comment
Understanding the domain and hardware context	Completed	Required to understand what the software receives and how the signal is interpreted.
Preparing the execution environment	Completed	The development environment was prepared under Windows with the required local dependencies.
Analyzing the legacy codebase	Completed at practical level	The key bottlenecks, couplings, and weak points were identified.
Refactoring the monolithic structure	Substantially completed	This became the central engineering task of the project.
Calibration functionality	Functional	The calibration tab is already operational in the current version.
Measurement functionality	In progress	The measuring tab is not fully working yet and remains one of the final integration tasks.
Packaging and executable generation	Completed	A reproducible packaging workflow was added with instructions in `README.md`.

6 Methodology and Development Environment

The methodology followed in the TFG has been incremental and iterative. Each cycle focused on a concrete technical problem, such as extracting logic from the main file, reorganizing startup behavior, clarifying data flow, or fixing GUI-to-backend interactions. This approach was more appropriate than a linear sequence because the project combines legacy-code analysis, hardware constraints, progressive testing, and frequent design decisions based on newly discovered issues.

A relevant practical part of the work was the preparation of the development environment. Python 3.12 was selected as a compromise between using a modern GUI stack based on PyQt6 and keeping a reasonably stable environment for the National Instruments drivers and the rest of the project dependencies. This choice was intended to modernize the interface side of the project without taking unnecessary risks with very recent versions of the full software stack.

The environment setup was more difficult than expected. Normal installation of dependencies through command-line tools was blocked by ICN2 network policies, so the required libraries and packages had to be searched manually one by one, checking each time that the selected file matched Windows and Python 3.12. After downloading them locally, the packages were installed through PyCharm from local files. This setup effort consumed approximately one working day of the project.

The project also includes a packaging workflow based on PyInstaller. This addition is useful because it allows future users to generate an executable version of the application by following a short documented procedure in `README.md`, instead of depending on PyCharm every time they need to run the software.

7 Current Software Structure

The refactored version of the software is being organized around clearer functional areas. Instead of concentrating all behavior in a single file, the current design separates at least startup logic, session control, acquisition access, signal processing, calibration-related behavior, GUI presentation, persistence, and packaging support. This decomposition is one of the main software engineering contributions of the TFG.

Table 4: Current functional structure of the software

Area	**Responsibility**
Application startup	Initializes the program, loads configuration, and connects the main workflow.
Session control	Coordinates the lifecycle of a session, including initialization, start, stop, and restart.
Hardware/acquisition layer	Communicates with the NI-based acquisition environment and obtains raw measured samples.
Signal-processing layer	Converts raw acquired values into processed numerical outputs suitable for interpretation.
Calibration-related logic	Runs or stores the calibration parameters required before or during measurement.
GUI layer	Presents controls, plots, execution status, and user feedback.
Persistence/export layer	Saves configurations, session data, and outputs for later inspection.
Packaging/support layer	Allows future users to build an executable application from the codebase.

8 Detailed Data Flow and Signal Representation

A stronger explanation of the data flow is necessary because one of the main responsibilities of the software is to acquire and transform the signals measured by the optical system. In the BiMW setup, the optical output is recorded by a two-sectional photodetector that provides two real-time intensity values, denoted as I_{up} and I_{down} [3]. These two magnitudes are central to the software because they are among the first meaningful values acquired from the hardware and they are directly shown, transformed, and used during processing.

Under the non-modulating regime described for the system, the sensitivity parameter SR is defined as [3]:

$$SR = \frac{I_{\text{up}} - I_{\text{down}}}{I_{\text{up}} + I_{\text{down}}} = A + B\cos(\phi) \tag{1}$$

This equation is important in the context of the software because it connects the raw photodiode intensities with the interferometric phase-related information that the system ultimately wants to interpret. In other words, the software does not simply display raw voltages: it acquires the two intensity-related detector signals, computes or uses them to derive the SR signal, and then uses the corresponding calibration and processing routines to obtain a more interpretable output.

The practical data flow can be summarized as follows. First, configuration and session parameters are loaded. Second, the software checks that the hardware environment is available. Third, the National Instruments layer acquires the digitized signals coming from the upper and lower sections of the photodiode. Fourth, those signals are organized into software-level numerical data structures. Fifth, the processing logic applies the transformations required to obtain meaningful traces such as I_{up}, I_{down}, SR, and the derived calibrated response. Sixth, the processed information is sent to the GUI for visualization and, when required, to the persistence layer for saving.

Table 5: Detailed software data flow

Stage	**Input**	**Output / role**
Session initialization	Configuration files and GUI parameters	Session state and runtime settings
Hardware verification	NI device availability and setup state	Confirmation that acquisition can begin
Acquisition	Analog detector signals from the photodiode	Digitized raw sampled data
Signal organization	Raw sampled arrays	Channel-associated data structures ready for processing
Intensity processing	Raw detector channels	I_{up} and I_{down} traces
Interferometric processing	I_{up}, I_{down}	Computed SR and other derived values
Calibration application	Calibration parameters and processed data	Corrected / interpretable output
Visualization and saving	Processed results	GUI plots, status indicators, and saved session data

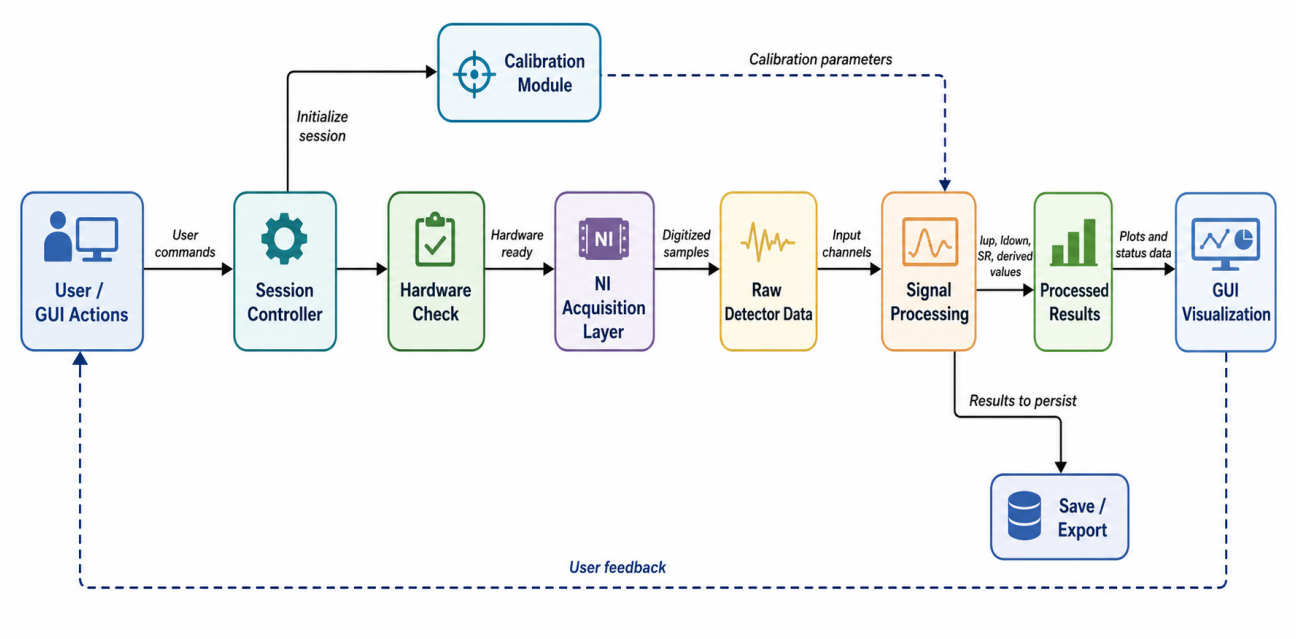

Figure 3: Detailed software and data-flow diagram.

9 Current Functionality, Tests, and Remaining Issues

The current software is partially functional. At the moment, the calibration tab works correctly and represents one of the most stable parts of the new implementation. In contrast, the measuring tab is still not fully functional and remains one of the main pending tasks before the project can be considered fully integrated.

An important result obtained during development is that the tests currently implemented for the different isolated parts of the new code pass correctly. This suggests that a significant part of the remaining problems are not caused by fundamentally wrong low-level logic, but by integration issues between components. In particular, one recurring problem is that certain actions triggered from the GUI do not produce the expected result because the corresponding backend method is not being called properly, because an old stub is still present, or because an exception causes the action to fail silently.

This distinction is important from an engineering perspective. The current bottleneck is not that the whole new architecture is failing, but that some interactions between interface and backend still need to be connected and cleaned up. For example, a button may appear to do nothing even though the underlying logic is already implemented, simply because the signal-slot connection is missing, incorrect, or still points to an incomplete placeholder implementation.

Table 6: Current functionality status

Component / functionality	**Status**	**Comment**
Application startup	Working	Startup time is now approximately 2 seconds.
Calibration tab	Working	Current stable functional part of the new application.
Measuring tab	Not fully working yet	Main pending area for final integration.
Isolated code tests	Passing	Implemented tests for separate parts of the code pass correctly.
GUI-to-backend integration	In progress	Some actions fail because methods are not connected or are still linked to stubs.
Silent failure diagnosis	In progress	Some failures are due to calls not being triggered properly or to exceptions not made visible enough.
Executable generation	Working	The project can be packaged following the documented `README.md` instructions.

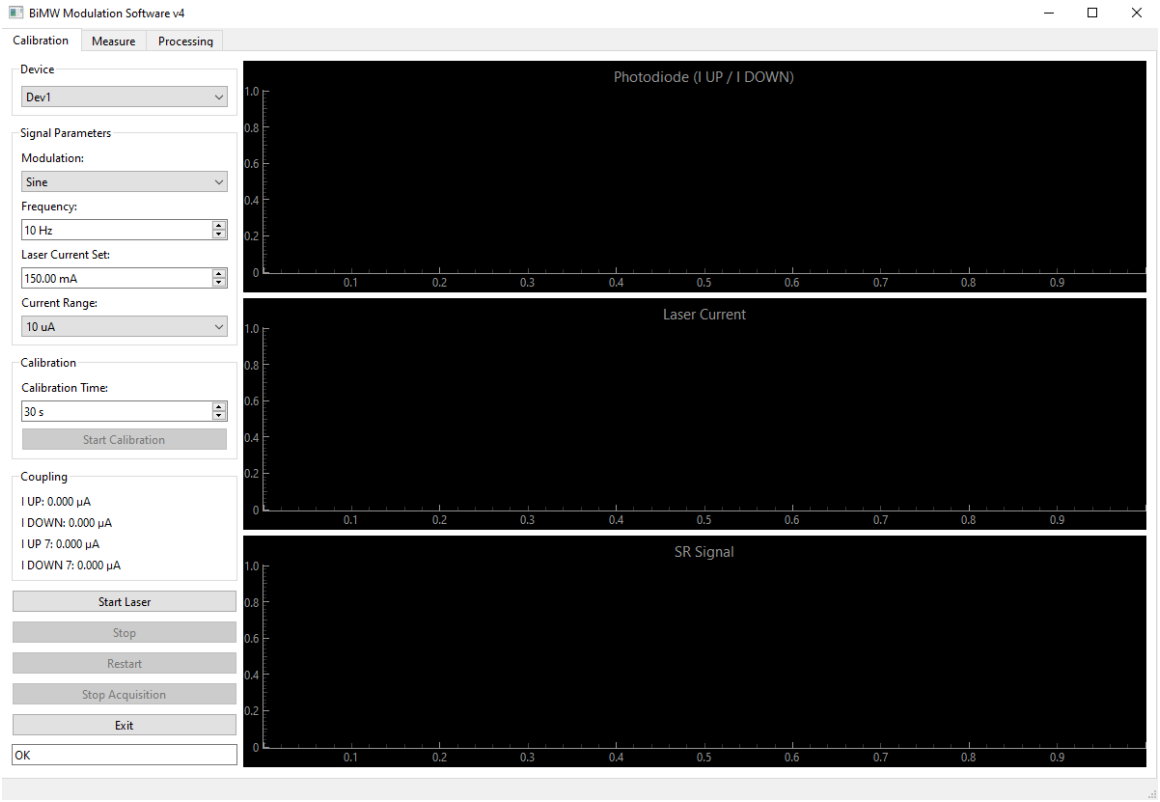

Figure 4: Screenshot of the current software interface, showing the calibration tab of the refactored application.

10 Results

The results obtained in this phase are mainly software-engineering results, but they are concrete and relevant. First, the project has substantially improved the original monolithic architecture and replaced it with a structure that is easier to understand and evolve. Second, the startup time has been reduced from approximately 30 seconds to approximately 2 seconds, which is an immediate improvement in day-to-day usability.

Third, the project now includes a reproducible workflow to generate an executable application. This reduces dependence on the development environment and makes the software more practical for future users who only need to run the program rather than modify it. Fourth, the data path from detector acquisition to processed visualization is now much more explicit, which improves debugging, documentation, and future reuse.

The current results must also be interpreted with realism. The new version is not yet fully complete because the measuring tab still requires final integration work. However, the fact that the calibration functionality is already operational and that the implemented tests pass for isolated parts of the code indicates that the remaining work is concentrated on integration and method-calling issues rather than on a failure of the redesign.

11 Provisional Conclusions

At the time of this report, the TFG is in its final development phase and has already produced meaningful improvements. The main conclusion is that the original problem was fundamentally architectural: concentrating most of the logic in a single main file made the software difficult to maintain, extend, and debug. The refactoring carried out in this project addresses that problem directly.

A second conclusion is that the new software already offers visible practical benefits. The faster startup time and the possibility of generating an executable version make the application more usable in a real laboratory context. A third conclusion is that the remaining problems are mainly integration problems, especially in the connection between GUI actions and backend methods, rather than evidence that the new design is conceptually wrong.

Overall, the project has substantially achieved its main software engineering goals. The final effort now has to focus on completing the measurement workflow, removing or replacing the remaining stubs, and ensuring that every relevant interface action calls the correct backend logic in a robust and explicit way.

12 Use of Generative AI

Generative AI tools were used only as limited support for language revision, translation, text restructuring, and clarity improvement in the preparation of this report. They were not used to replace the core intellectual work of the project.

The software analysis, the identification of the monolithic-architecture problem, the engineering decisions, the refactoring work, the interpretation of the results, and the conclusions presented in this report were carried out by the author. The use of AI was restricted to support tasks and not to the generation of the technical content that is subject to evaluation.

13 Sources of Information Consulted

The development of this TFG has required combining several types of sources. First, scientific and technical references related to the BiMW platform and its read-out algorithm were necessary to understand what is being measured and how the SR signal is defined. Second, project-specific material, including the legacy codebase and previous TFG reports, was necessary to identify the actual software architecture problems that motivated the refactoring. Third, software-tool documentation was useful to support packaging and deployment decisions.

References

[1] M. Soler and L. M. Lechuga, *Label-Free Photonic Biosensors: Key Technologies for Precision Diagnostics*, ChemistryEurope, 2025.

[2] L. M. Lechuga, *Tecnología de biosensores para el diagnóstico descentralizado*, 2021.

[3] B. Bassols-Cornudella, P. Ramírez-Priego, M. Soler, M.-C. Estevez, H. J. Díaz Luis-Ravelo, M. Cardenosa-Rubio, and L. M. Lechuga, "Novel Sensing Algorithm for Linear Read-Out of Bimodal Waveguide Interferometric Biosensors," *Journal of Lightwave Technology*, vol. 40, no. 1, 2022.

[4] K. E. Zinoviev, A. B. González-Guerrero, C. Domínguez, and L. M. Lechuga, "Integrated Bimodal Waveguide Interferometric Biosensor for Label-Free Analysis," *Journal of Lightwave Technology*, vol. 29, no. 13, 2011.

[5] Escola d'Enginyeria, Universitat Autònoma de Barcelona, *Avaluació del Treball Final de Grau / GEI-AvaluacioTFG*, course documentation, 2025–2026.

[6] PyInstaller Development Team, *PyInstaller Manual*, available at: https://www.pyinstaller.org

[7] National Instruments, *Using a NI DAQ Device with Python and NI DAQmx in Windows*, available at: https://knowledge.ni.com/KnowledgeArticleDetails?id=kA00Z000000P8o0SAC